Experiment 3: Exact-Scan vs. Tree-Indexed Nearest-Neighbor Search for SAW's Semantic Memory

Status: completed, measured experiment. This tests the core retrieval operation behind SAW's pgvector-based semantic-memory layer — nearest-neighbor search over high-dimensional embeddings — which had never been benchmarked before. It is the third experiment in this project, alongside SEAG (approval-gate governance) and Experiment 2 (audit-trail query patterns).

Critical scope note — read before interpreting any number below

This experiment does not run actual pgvector, and does not use a real embedding model. Neither could be installed or downloaded in the sandboxed environment used to run this benchmark (no network access to install pgvector or download a sentence-embedding model). Instead, it isolates the underlying operation pgvector's indexes exist to accelerate — nearest-neighbor search over high-dimensional vectors — using available substitutes:

Represents	Actual tool used	Why it's a fair proxy
Real text embeddings	Synthetic unit-normalized random vectors (384 dimensions, matching typical sentence-embedding size)	Same shape and comparison operation (cosine similarity over ~300-dim vectors) as real embeddings, though carrying no real semantic content
Unindexed pgvector column (sequenti	NumPy brute-force cosine similarity	Same exhaustive-scan operation

Represents	Actual tool used	Why it's a fair proxy
al scan)		
Indexed pgvector column (e.g., IVFFlat/HNSW)	scikit-learn's BallTree and KDTree	Conceptually the same purpose (accelerate nearest-neighbor search via an index), though not the same algorithm as pgvector's actual index types

Any number below describes **exact-scan vs. tree-indexed search behavior on synthetic vectors**, not a benchmark of pgvector or of real semantic retrieval quality.

Motivation

SAW's architecture designs a pgvector-based semantic-memory layer for its seven-agent orchestration pipeline, but the actual retrieval operation — finding the most relevant stored memory items for a given query — had never been tested, even after Experiment 2 tested a simpler audit-trail query. This experiment closes that gap by testing the nearest-neighbor search itself, rather than continuing to treat it as a solved problem by assumption.

Method

- **Synthetic data:** 50,000 unit-normalized random vectors in 384 dimensions, representing stored agent-memory embeddings.
- **Random seed:** 42, fixed for full reproducibility.
- **Query:** top-5 nearest-neighbor retrieval (the standard "find the most relevant memories" operation) for 50 randomized query vectors.
- **Methods compared:**

    - **Brute-force exact cosine similarity** (NumPy) — full scan of all 50,000 vectors per query.
    - **BallTree** (scikit-learn) — a spatial index built once over all 50,000 vectors, then queried.
    - **KDTree** (scikit-learn) — a second spatial index, same build-once-query-many pattern.
- **Correctness check:** for each of the 50 queries, verified that all three methods returned the identical top-5 set of nearest neighbors before comparing timing. All 50 queries agreed exactly.
- Index build time was measured separately from per-query time, since a real database builds an index once and reuses it for many subsequent queries.

Results (two independent runs, milliseconds per query, n = 50 queries, top-5)

Method	Run 1 mean (ms)	Run 2 mean (ms)	Index build time
Brute-force (exact scan)	6.66	7.59	— (no index)
BallTree	18.76	20.92	~1.4–1.5s
KDTree	24.33	26.75	~2.0–2.2s

Both runs agree qualitatively: the tree-indexed methods were consistently **2.7–3.5× slower per query** than the unindexed brute-force scan, in addition to costing 1.4–2.2 seconds to build the index in the first place.

Honest interpretation — a second negative result for the intuitive hypothesis

The intuitive expectation — carried over from Experiment 2's finding that indexing helped a row-oriented query — would be that an index should also help nearest-neighbor search. **It did not, here.** This is not a

contradiction or an error; it reflects a well-documented phenomenon in similarity search known as the “curse of dimensionality”: tree-based exact nearest-neighbor structures (KDTree, BallTree) lose their advantage over brute-force search as vector dimensionality grows, because in high dimensions most of the tree’s branches end up needing to be checked anyway. At 384 dimensions, this project’s synthetic data sits well within the range where this effect is well known to appear.

Practical implication for SAW: this result does not mean pgvector’s own indexes (IVFFlat, HNSW) would behave the same way — those are *approximate* nearest-neighbor methods specifically designed to avoid this degradation, unlike the *exact* tree methods tested here. What this experiment does show is that assuming “any index helps” is not safe for high-dimensional vector search, and that if SAW’s semantic-memory layer is ever benchmarked against real pgvector, the comparison should specifically test pgvector’s approximate indexes rather than exact tree structures — because this experiment suggests exact indexing may not be the right tool for this job at these dimensions.

Explicit limitations

- 50,000 vectors is a moderate scale. The relative behavior of exact tree indexes vs. brute force can shift at very different scales, and this was not tested across multiple sizes.
- Only Euclidean-equivalent nearest-neighbor search on unit-normalized vectors was tested (mathematically equivalent to cosine similarity here, but this equivalence does not extend to all distance metrics).
- Approximate nearest-neighbor methods (which is what pgvector actually uses in production) were not tested — only exact methods (brute-force, BallTree, KDTree) were available without network access to install specialized libraries (e.g., FAISS, HNSWlib).
- As stated above, this is not a benchmark of pgvector itself, nor of real embedding quality or retrieval relevance — it isolates the geometric search-cost question using available local tools.

Reproducibility

Full code (`experiment3.py`) and raw results (`experiment3_results.json`) are included alongside this report. Running `python3 experiment3.py` with the same seed (42) regenerates the same vectors and queries; per-query timings vary slightly with hardware and system load (as reflected in the two-run comparison above), but the qualitative result — tree-indexed exact search being slower than brute force at this dimensionality — was confirmed consistent across independent runs.